

dev tra Git編

チーム制作のためのGit入門（全14回）

【1-01】Gitってなに？なんで必要？【1章：Git導入編】

1. クリア条件：Gitが何をしてくれるものか、ざっくり分かれればOK！

2. ここからは別巻、チーム制作のためのGit編です。

今回は手を動かしません。まず「なんでこれを覚えるの？」をハッキリさせておきましょう。ここが分かっていると、この先の操作が意味不明な暗記になってしまうので。

3. まず、こんな経験ありませんか？

```
キャラ絵_最終.psd  
キャラ絵_最終2.psd  
キャラ絵_最終_修正版.psd  
キャラ絵_ほんとの最終.psd  
キャラ絵_ほんとの最終_これで決定.psd
```

あるあるですね。これは、ファイル名で変更履歴を管理している状態です。

そして大体、3日後には「どれが最新？」「あの時の色に戻したいけど、どのファイルだっけ？」となります。

4. Gitを使うと、ファイルは「キャラ絵.psd」1つだけになります。

そのかわり、キリのいいところで**その時点の状態を記録**していきます。この記録が積み重なって履歴になります。

用語：Git（ギット）

ファイルの変更履歴を記録・管理するソフト。ゲームのセーブに近く、**上書きではなく、記録するたびにセーブデータが1つずつ増えていきます**。だから、いつでも好きな時点の状態に戻せます。

5. でも、Gitが本当に効いてくるのはチーム制作のときです。

たとえば、AさんがタイトルUIを作り、Bさんが敵の動きのスク립トを書いているとします。Gitを使わずにUnityプロジェクトを共有すると、どうなるか。

- Aさん「プロジェクトフォルダをzipで送るね」
- Bさん「ありがとう。こっちも直したから送り返すね」
- Aさん「その間にこっちも直しちゃった。どっちが正しいの？」
- 二人「……」

zipを送り合う運用は、必ずどこかで誰かの作業が消えます。

6. Gitを使うと、こうなります。

- 全員が同じ場所（GitHub上のリポジトリ）のデータを見る

- 各自が自分の作業をして、終わったらそこに送る
 - Gitが「AさんはUIのファイル、Bさんはスクリプトのファイルを変えたんだな」と判断して、**自動で1つにまとめてくれる**
- 触っているファイルが違う限り、2人の作業は自動的に1つにまとまります。ここがGitの一番の強みです。

用語：マージ

別々に進めた作業を1つにまとめること。ほとんどの場合、Gitが自動でやってくれます。

7. まとめると、Gitがやってくれるのはこの3つです。

- **変更履歴を全部残す**（いつでも過去の状態に戻せる）
- **複数人の作業を1つにまとめる**（zipの送り合いが不要になる）
- **誰がいつ何を変えたかを記録する**（作業ログが自動で残る）

8. 「絵や音を作るだけなら関係ないのでは？」と思うかもしれませんが、Unityではそうもいきません。

画像を1枚追加するだけでもプロジェクトのファイルが増えますし、シーンの設定も書き換わります。絵もアニメも音もスクリプトも、全部まとめてGitの管理下に入ります。

このサークルでは全員が同じプロジェクトを触るので、**全員が使えるようになっておく必要があります**。

9. 最後に、心構えをひとつ。

Gitは最初、用語が多くて戸惑うと思います。コミット、プッシュ、ブランチ、マージ.....たぶん何度か「わからん」となります。

でも安心してください。実際に毎日使う操作は、この4つだけです。

- **コミット**（変更を記録する）
- **プッシュ**（記録をGitHubに送る）
- **プル**（GitHubから最新を取ってくる）
- **ブランチ**（作業する場所を分ける）

残りは全部この応用です。操作はすべて、マウスで使えるアプリ（1-04で入れます）から行います。

10. Gitが「履歴が全部残る、みんなで使える保存場所」だとイメージできたら、今回は完成です！

次回は、その保存場所を用意するところから始めます。

【1-02】GitHubアカウントを作ろう【1章：Git導入編】

1. クリア条件：GitHubのアカウントを作って、ログインできればOK！

2. 前回、Gitは「履歴が全部残る保存場所」だと説明しました。

では、その保存場所をチーム全員でどう共有するのか？　そこで使うのがGitHub（ギットハブ）です。

用語：GitとGitHubの違い

よく混同されるので、最初にハッキリさせておきます。

Git = 自分のPCで動くソフト。手元のファイルの変更履歴を記録する

GitHub = インターネット上のサービス。Gitで記録した履歴を置いて、チームで共有する

つまり、**手元で記録するのがGit、それを全員で共有する場所がGitHub**です。両方そろって初めてチーム制作に使えます。

3. GitHubは無料で使えて、世界中の開発現場で使われています。マスコットは猫とタコを合体させた「Octocat」で、サイトのあちこちに出てきます。

4. では作りましょう。ブラウザで下のURLを開いてください。

<https://github.com/signup>



アクセスは一時的に制限されています

これが表示されたのは、ブラウザによる不審な挙動が検知されたためです。

以下のような理由が考えられます：

- ページ遷移やクリックが異常に速い
- お使いのコンピュータ上でJavascriptの機能がブロックされている
- 同じネットワーク(IP 153.240.242.132)上にロボットがある

サイトへのアクセスに問題がありますか？ [サポートに連絡する](#)

ID: 19302724-11ca-3d9b-3226-95e8bda1eba4

5. 入力するのは3つだけです。

- **Email**：普段使っているメールアドレス
- **Password**：15文字以上、または8文字以上 + 数字と小文字を含む
- **Username**：あなたのID。**チームメンバーの画面にも表示される名前**なので、普段呼ばれている名前がおすすめ

※Usernameはあとから変更できますが、URLが変わって面倒なので最初に決めてしまいましょう。半角英数とハイフンのみ使えます。

6. 「Country/Region」は Japan を選択。メール配信の希望はチェックしなくてOKです。

7. 「Create account」を押すと、メールに6桁の確認コードが届きます。入力すれば完了です。

※届かないときは迷惑メールフォルダを確認してください。

8. そのあと「What do you want to do first?」のようなアンケート画面が出ますが、全部スキップして大丈夫です。「Skip personalization」や「Continue for free」を選びましょう。

※有料プランを勧められますが、**Freeで十分**です。

9. ログインして自分のトップページが表示されたら成功です！

まだ中身は空っぽですが、これでチームのプロジェクトに参加する準備ができました。

10. 最後に、**自分のUsernameをチームの代表の人に伝えておきましょう。**

代表の人が、あなたを「このプロジェクトを編集できる人」として招待するのに必要です。招待されないと、GitHubにデータを送る権限がもらえません。

※Discordなどで「GitHubのIDは〇〇です」と送っておけばOKです。

11. GitHubにログインできて、IDをチームに伝えたら、今回は完成です！

【1-03】Gitをインストールしよう【1章：Git導入編】

1. クリア条件：自分のPCにGit本体が入って、名前とメールアドレスの設定ができればOK！

2. 前はGitHub（インターネット側）の準備をしました。今回は自分のPCにGit本体を入れます。

Git本体は、変更履歴を記録する処理そのものを担当します。次回入れるアプリ（Fork）は、このGit本体を裏側で呼び出して動く操作画面です。

つまり、Git本体が入っていないとForkは動きません。順番として、先にこちらを入れます。

3. まずは、すでに入っていないか確認しましょう。UnityやVisual Studioを入れたときに、一緒に入っていることがあります。

Windowsの人：スタートメニューを開いて「Git Bash」と検索してください。



スタートメニューでGit Bashを検索

このように「Git Bash」というアプリが出てきたら、すでに入っています。→ 手順8へ進んでください。

何も出てこなければ、まだ入っていません。→ 手順4へ。

Macの人：手順7を見てください。

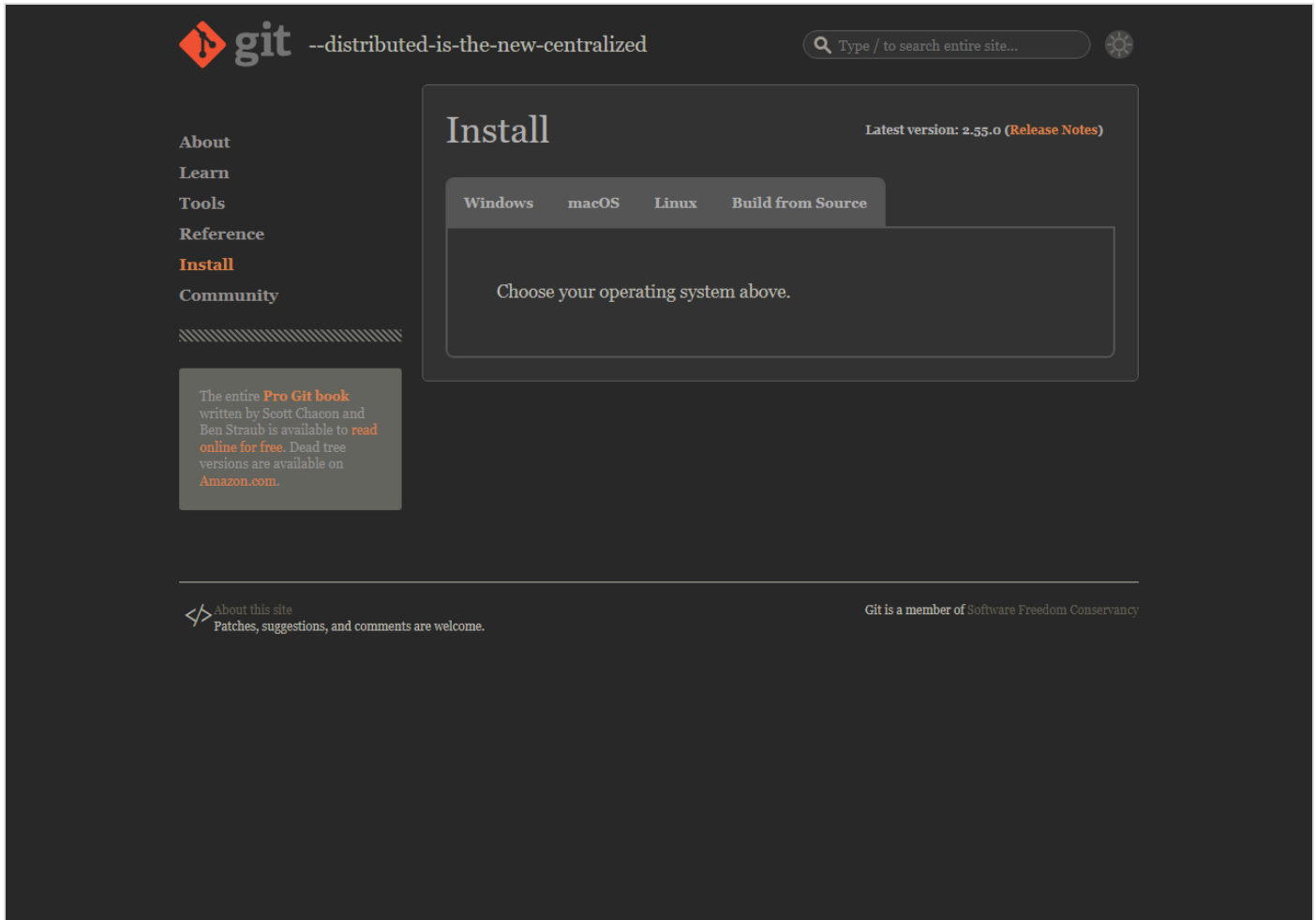
用語：Git Bash

Gitを入れると一緒に付いてくる、文字を打ってPCに命令するためのアプリです。Windowsの「コマンドプロンプト」の仲間だと思ってください。

このチュートリアルでGit Bashを使うのは、今回の確認と設定だけです。次回以降の作業は全部マウス操作でやります。

4. 入っていなかった人は、公式サイトからダウンロードします。

<https://git-scm.com/downloads>



Gitの公式ダウンロードページ

※黒背景に赤いロゴの、ちょっと無骨なサイトです。合っています。

5. 【Windowsの人】「Windows」を選び、「Click here to download」から最新版をダウンロード。
ダウンロードした .exe をダブルクリックして進めます。
6. 【Windowsの人】インストーラーの設定項目がたくさん出てきますが、**全部そのまま「Next」で大丈夫**です。標準の設定で問題なく動きます。
最後に「Install」→「Finish」で完了。
※途中で「デフォルトのエディタを選べ」という画面が出ます。ここもそのまま（Vim）で問題ありません。今回は使いません。
7. 【Macの人】Macは少し簡単です。「ターミナル」アプリを開いて、`git --version` と打ってEnterを押してください。
入っていなければ「開発者ツールをインストールしますか？」というダイアログが出るので、「インストール」を押すだけで入ります。
※Homebrewを使っている人は `brew install git` でもOKです。

8. 入ったかどうか確認しましょう。

Windowsは「Git Bash」、Macは「ターミナル」を開いて、下の1行を打ってEnterを押します。

```
git --version
```

`git version 2.55.0` のようにバージョン番号が表示されたら成功です！

9. 続けて、あなたの名前とメールアドレスを設定します。

Gitは変更履歴に「誰が記録したか」を残すので、この2つを最初に教えておく必要があります。設定しないと、あとで記録するときにエラーになります。

同じ画面で、下の2行を1行ずつ実行してください（`""` の中は自分のものに書き換え）。

```
git config --global user.name "あなたの名前"
```

```
git config --global user.email "GitHubに登録したメールアドレス"
```

※名前は日本語でも動きますが、ローマ字がGitHubのIDにしておくのと、あとで履歴を見たときに読みやすいです。

※メールアドレスは1-02でGitHubに登録したものにしてください。ここが違くと、GitHub上で「誰の作業か」が正しく表示されません。

10. 設定できたか確認するには、下の1行を打ちます。設定した名前とメールアドレスが表示されればOKです。

```
git config --global --list
```

11. お疲れさまでした。文字を打つ操作はここまでです。

次回、マウス操作でGitを使うアプリを入れたら、そこから先はボタン操作だけで進みます。

12. バージョン番号が表示されて、名前とメールアドレスの設定ができれば、今回は完成です！

【1-04】Forkを入れよう（マウスでGitを使う）【1章：Git導入編】

1. クリア条件：GitのアプリForkを入れて、GitHubアカウントと連携できればOK！
2. 前回入れたGit本体だけだと、記録や送信のたびに文字を打つことになります。毎日やるには大変なので、**マウスで操作できるアプリ**を入れます。

用語：Gitクライアント

Gitの操作をボタンやアイコンでできるようにしたアプリのこと。裏でやっていることはGit本体と同じなので、**できることも結果も文字入力の場合と全く同じ**です。

3. 今回使うのは **Fork** というアプリです。理由は3つ。
 - 変更履歴が**枝分かれの図**で表示されるので、誰がどこで何をしたか目で追える
 - 変更されたファイルの中身を、画像はサムネイル付きで確認できる
 - 動作が軽い

※Windows版とMac版があり、**画面の作りがほぼ同じ**なもの採用理由です。チーム内でOSが混ざっていても、同じ説明が通じます。

4. では入れましょう。下のURLを開いてください。

<https://git-fork.com/>



Fork is getting better and better day after day and we are

Forkの公式サイト

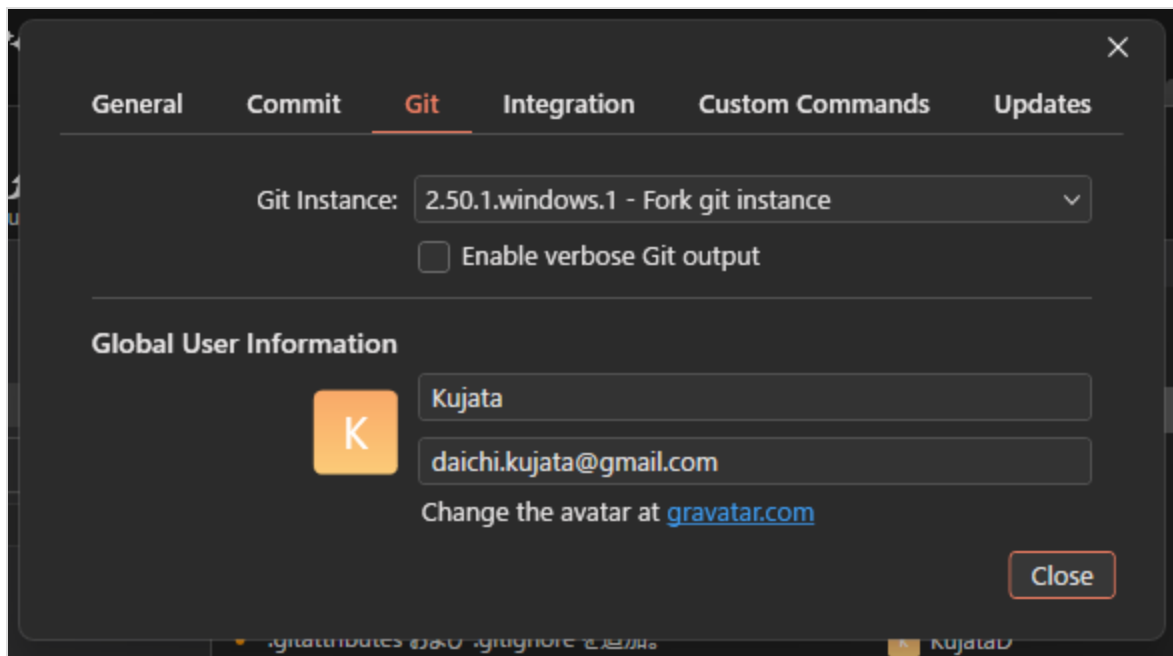
自分のOSのボタン（Download for Windows / Download for Mac）を押してダウンロード、インストールします。

※Forkは「期限なしで試用できる有料アプリ」です。起動のたびに購入案内が出ますが、機能制限なく使い続けられます。気に入ったら購入を検討してください（買い切り \$59.99）。

5. 初回起動時に、名前とメールアドレスを聞かれます。1-03で設定したものと同じものを入れてください。

（1-03で設定済みなら、自動で入っているはずです）

あとから確認・変更するときは、**File > Preferences** を開いて「Git」タブを見ます。この「Global User Information」が、1-03で設定した内容です。



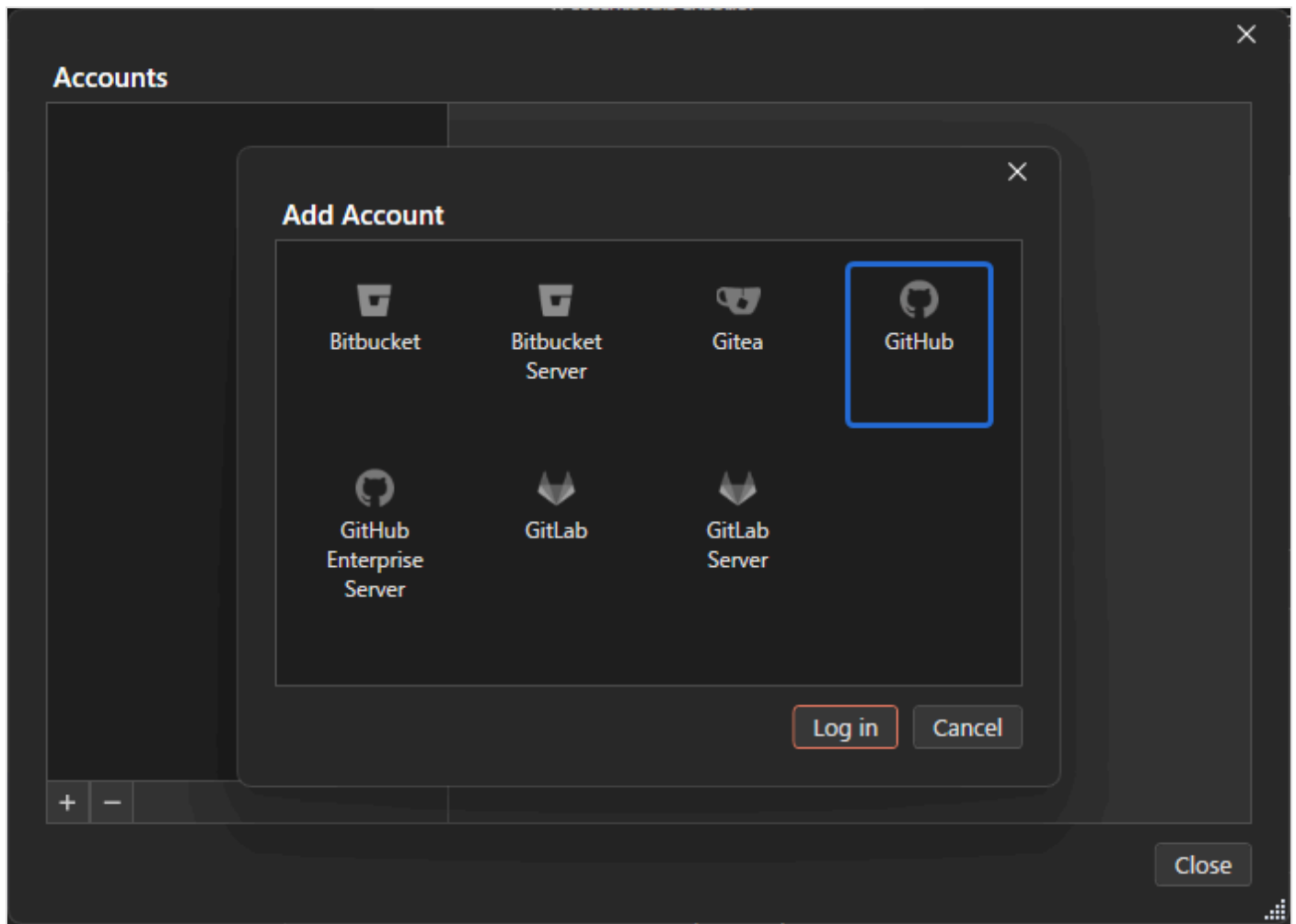
Forkの名前・メールアドレス設定

6. 次に、GitHubアカウントと連携します。連携しておくと、GitHubにデータを送るたびにパスワードを聞かれずに済みます。

- Windows : File > Accounts...
- Mac : Fork > Accounts...

を開いてください。

7. 左下の「+」ボタンを押すと、サービスの一覧が出ます。GitHubを選んで「Log in」を押しましょう。



ForkのAdd Account画面

8. ブラウザが開いてGitHubの認証画面が表示されるので、緑色の「Authorize fork」ボタンを押せば連携完了です。
Forkの画面に自分のGitHubアカウント名が表示されていれば成功です。

9. 【補足】VSCodeを使っている人へ

Visual Studio Code にも、Gitの機能が最初から入っています（左端の枝分かれアイコン）。
スクリプトを書きながらそのまま記録できるので便利ですが、履歴の枝分かれが確認しづらいのが弱点です。
おすすめは併用です。「日々の記録はVSCode、ブランチ操作や履歴の確認はFork」という使い分けをしている人が多いです。もちろん全部Forkでも構いません。

10. Forkが起動して、GitHubと連携できたら、今回は完成です！

これで環境の準備は終わりです。次回からは実際のプロジェクトを触っていきます。

【1-05】Unity用の.gitignoreを知ろう【1章：Git導入編】

1. クリア条件：Gitで共有しないファイルがあることが分かって、Unity用の設定を用意できればOK！
2. 今回は操作より説明が多めですが、**ここを飛ばすとチーム制作でほぼ確実に事故ります**。5分だけお付き合いください。
3. まず、Unityのプロジェクトフォルダを開いてみてください。こんなフォルダが並んでいます。

Assets/	← 自分で作った素材・スクリプト
ProjectSettings/	← プロジェクトの設定
Packages/	← 使っている機能のリスト
Library/	← ★
Temp/	← ★
Logs/	← ★
obj/	← ★

4. ★の付いたフォルダは、**Unityが自動で作る一時ファイル**の置き場です。
特にLibraryは、Assetsに入れた画像や音をUnityが内部用に変換したデータが入っていて、数GBになることもあります。
5. ここで問題です。このLibraryフォルダは、チームで共有すべきでしょうか？

答えは**共有しない**です。理由は3つ。

- **サイズが大きい**：数GBを毎回アップロード・ダウンロードすることになる
- **共有する意味がない**：Assetsさえあれば、Unityが各自のPCで自動的に作り直してくれる
- **必ず食い違う**：全員のPCで中身が微妙に違うため、毎回「変更あり」と判定されて衝突の原因になる

つまり、**共有するのは自分で作ったもの（Assets、ProjectSettings、Packages）だけ**。Unityが自動生成するものは各自のPCで作らせる、が基本方針です。

6. そこで使うのが.gitignore（ギット・イグノア）です。

用語：.gitignore

「このファイルはGitで記録しなくていい」というリストを書いた設定ファイル。ignore（無視する）ものを指定します。ファイル名がドットから始まるので、フォルダ内で先頭のほうに表示されます。

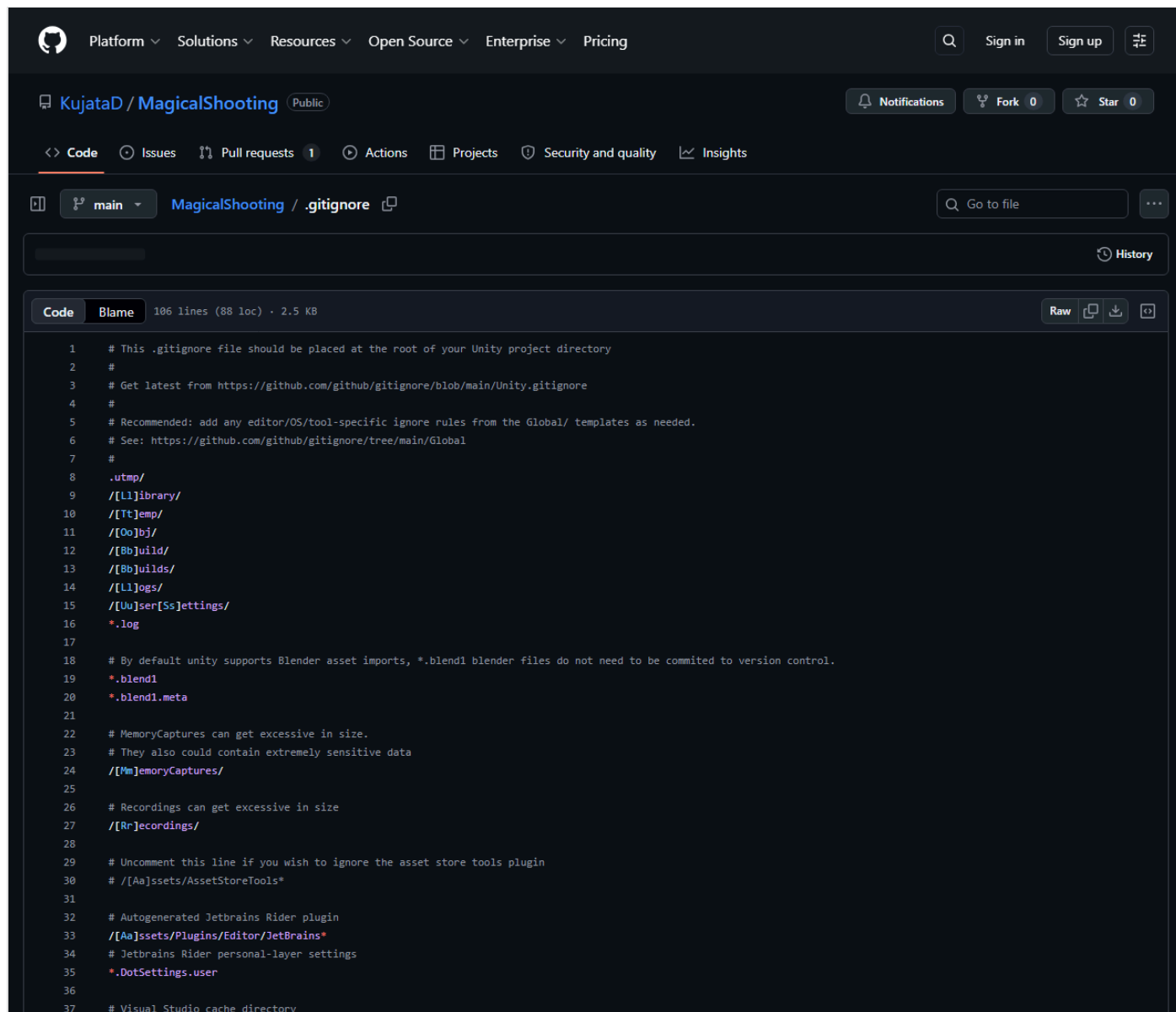
7. Unity用の.gitignoreは、**GitHubが公式のテンプレートを用意してくれています**。自分で書く必要はありません。
次回リポジトリを作るときに、選択肢から「Unity」を選ぶだけで自動生成されます。今回は中身を眺めるだけでOKです。

```
# Unityが自動生成するフォルダ（共有しない）
[LI]brary/
[Tt]emp/
[Oo]bj/
[Bb]uild/
[LI]ogs/
```

```
# ビルドの成果物（共有しない）
*.apk
*.unitypackage

# OSが自動で作るファイル（共有しない）
.DS_Store      ← Macが作る隠しファイル
Thumbs.db       ← Windowsが作るサムネイルの一時ファイル
```

実物はこんな感じです。100行ほどありますが、全部GitHubが用意してくれたものです。



```
1  # This .gitignore file should be placed at the root of your Unity project directory
2  #
3  # Get latest from https://github.com/github/gitignore/blob/main/Unity.gitignore
4  #
5  # Recommended: add any editor/OS/tool-specific ignore rules from the Global/ templates as needed.
6  # See: https://github.com/github/gitignore/tree/main/Global
7  #
8  .utmp/
9  /[Ll]ibrary/
10 /[Tt]emp/
11 /[Oo]bj/
12 /[Bb]uild/
13 /[Bb]uilds/
14 /[Ll]ogs/
15 /[Uu]ser[Ss]ettings/
16 *.log
17
18 # By default unity supports Blender asset imports, *.blend1 blender files do not need to be committed to version control.
19 *.blend1
20 *.blend1.meta
21
22 # MemoryCaptures can get excessive in size.
23 # They also could contain extremely sensitive data
24 /[Mm]emoryCaptures/
25
26 # Recordings can get excessive in size
27 /[Rr]ecordings/
28
29 # Uncomment this line if you wish to ignore the asset store tools plugin
30 # /[Aa]ssets/AssetStoreTools*
31
32 # Autogenerated JetBrains Rider plugin
33 /[Aa]ssets/Plugins/Editor/JetBrains*
34 # JetBrains Rider personal-layer settings
35 *.DotSettings.user
36
37 # Visual Studio cache directory
```

Unity用の.gitignore

8. 【超重要】Unity特有の注意点：.metaファイルは必ず一緒に記録する

Assetsフォルダを見ると、画像1枚につき「Player.png」と「Player.png.meta」の2つがあるはずです。

この.metaファイルには、その素材の設定（Pixels Per Unit、Sprite Modeなど）と、Unity内部での識別番号（GUID）が書かれています。

もし.metaを共有し忘れると、こうなります。

- 自分のPCでは、Playerオブジェクトに絵がちゃんと付いている
- 相手のPCでは、**絵が外れて白い四角（Missing）**になる

Unityでよくある事故です。「相手の環境だけ絵が消える」の原因は、だいたいこれ。

Forkを使っていれば、.metaも一覧に表示されるので、**変更されたファイルは全部まとめて記録する**と覚えておけば大丈夫です。

9. 【もうひとつ重要】シーンとプレハブは同時に触らない

.gitignoreの話とセットで覚えてほしいのがこれです。

Gitは、テキストファイルなら「Aさんは3行目、Bさんは50行目を変えた」と判断して自動でまとめられます。

でもUnityのシーンファイル（.unity）やプレハブ（.prefab）は中身が複雑なテキストなので、**同じシーンを2人が同時に編集すると、ほぼ確実に自動でまとめられなくなります**（これをコンフリクトと呼びます。1-13で扱います）。

なので、チームでの決めごとはこの3つです。

- **同じシーン・同じプレハブを、同時に2人で触らない**
- 誰がどのシーンを触るか、作業前にDiscordで宣言する
- 大きなシーンは、機能ごとにプレハブに分けて分担する

※これは1-14の作業マニュアルにも載せます。忘れてもそちらを見ればOKです。

10. 「Libraryは共有しない」「.metaは一緒に記録する」「同じシーンを同時に触らない」の3つが頭に入ったら、今回は完成です！

【1-06】リポジトリを作ろう【1章：Git導入編】

1. クリア条件：GitHubにチーム用のリポジトリを作って、メンバーを招待できればOK！
2. いよいよプロジェクト用の置き場をGitHubに作ります。この置き場をリポジトリと呼びます。

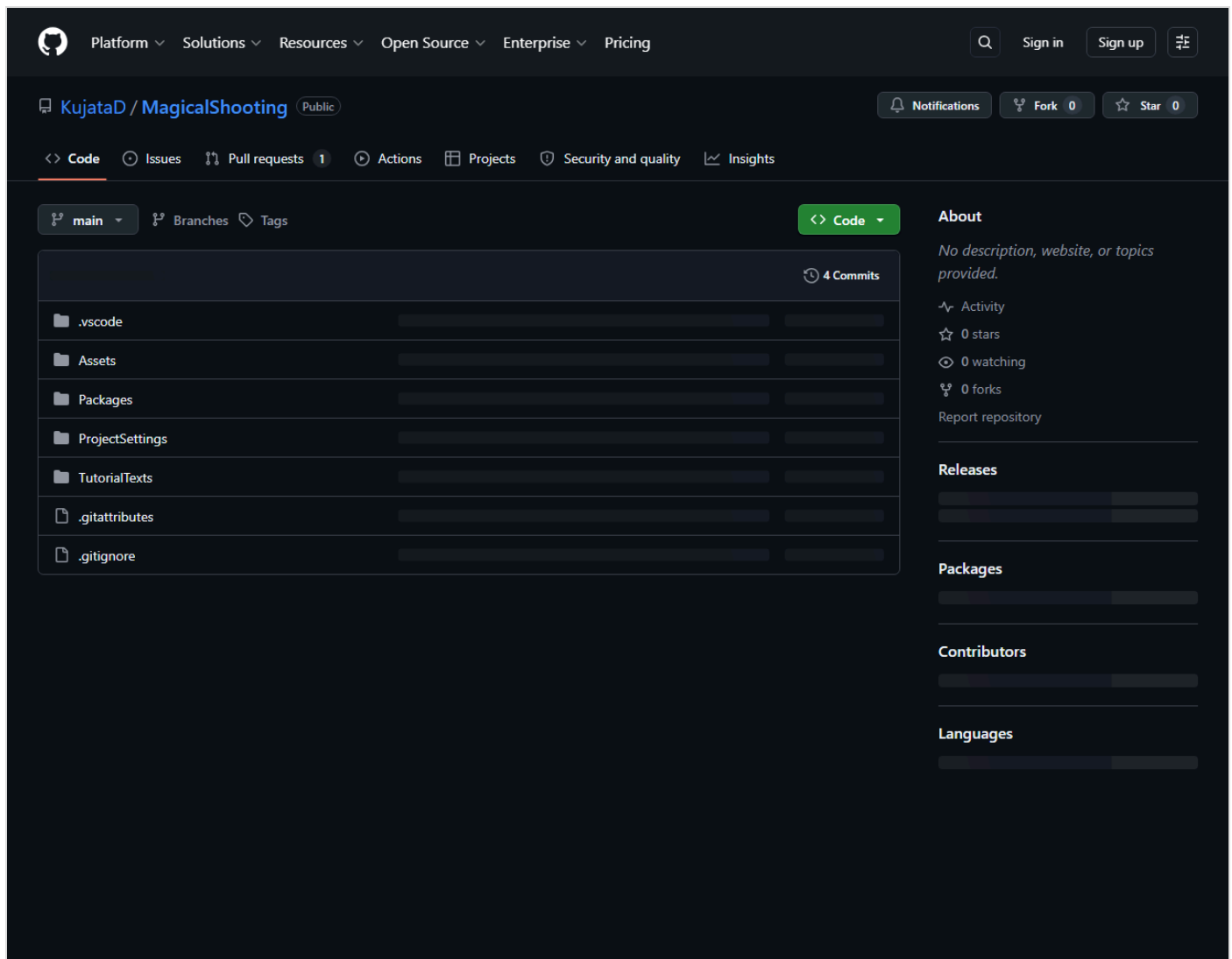
用語：リポジトリ（repository）

プロジェクト1つ分のファイルと、**その全変更履歴**が入っている置き場。プロジェクト1つにつきリポジトリ1つ、と考えてください。

3. ※この作業は**チームで1人（代表の人）**だけがやればOKです。他のメンバーは眺めるだけで大丈夫。自分が代表でない人は、次の「クローン」から参加になります。
4. GitHubにログインした状態で、右上の「+」→「New repository」を選びましょう。
5. 設定項目を埋めていきます。
 - **Repository name**：プロジェクト名（例：MagicalShooting）。半角英数とハイフン推奨
 - **Description**：どんなゲームか一言（任意）
 - **Public / Private**：
 - Public＝インターネット上の誰でも見られる
 - Private＝招待した人だけ見られる

練習ならどちらでもOK。素材の権利が気になるならPrivateが安全です
6. 下にスクロールして、ここが**今回いちばん大事**なところです。
 - **Add a README file**：チェックを入れる（プロジェクトの説明書。あとで書けます）
 - **Add .gitignore**：ドロップダウンから **Unity** を選ぶ ← 前回説明したものです
 - **Choose a license**：チーム内制作なら「None」でOK

.gitignoreで「Unity」を選び忘れると、Libraryフォルダまで全部共有されてしまい、前回説明した問題がそのまま起きます。ここだけは必ず確認してください。
7. 「Create repository」を押すと、リポジトリが作成されます！



作成されたリポジトリ

※このスクショは、すでに中身が入った状態のもので。作りたてはREADME.mdと.gitignoreだけが入っています。

8. 次に、チームメンバーを招待します。これをやらないと、メンバーは中身を見られてもデータを送れません（プッシュできません）。

- リポジトリの上部メニューから「Settings」を開く
- 左メニューの「Collaborators」（共同編集者）を選ぶ
- 「Add people」を押して、メンバーのGitHub IDを入力

1-02でメンバーに「IDを教えてね」と伝えておいたのは、このためでした。

9. 招待されたメンバーには、GitHubからメールが届きます。中の「Accept invitation」を押してもらえば参加完了です。

※招待は7日で期限切れになるので、メンバーには「招待を送ったので確認してね」と一声かけましょう。

10. 最後に、リポジトリのURLをチーム全員に共有しましょう。次回、全員がこのURLを使います。

`https://github.com/ユーザー名/リポジトリ名`

11. リポジトリができて、メンバーが参加できたら、今回は完成です！
次回は、このリポジトリの中身を自分のPCに持ってきます。

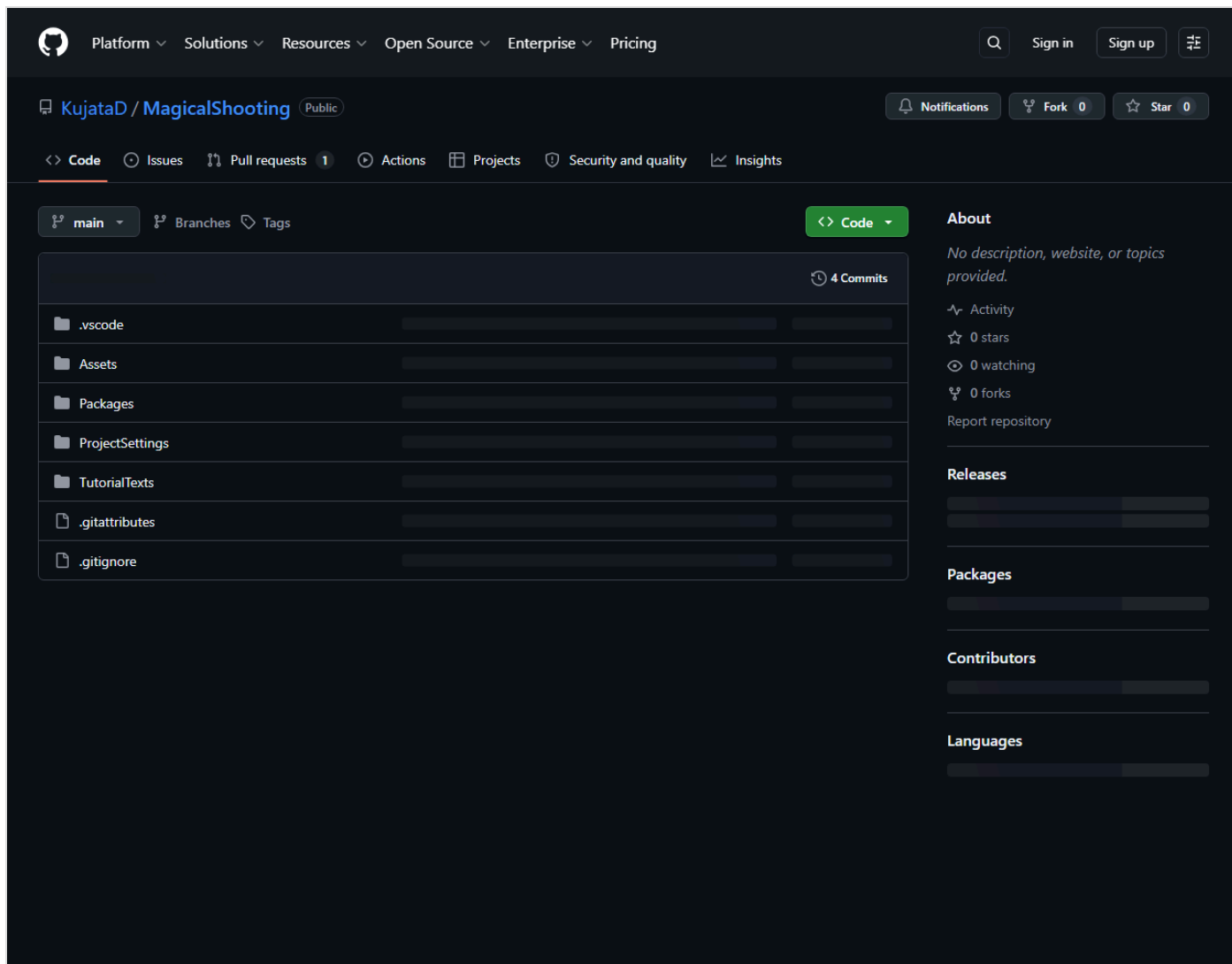
【1-07】クローンしよう（手元に持ってくる）【1章：Git導入編】

1. クリア条件：GitHubのリポジトリを自分のPCにコピーして、Unityで開ければOK！
2. リポジトリはできましたが、まだGitHub上にあるだけです。作業するには自分のPCに持ってくる必要があります。この操作をクローンと呼びます。

用語：クローン（clone）

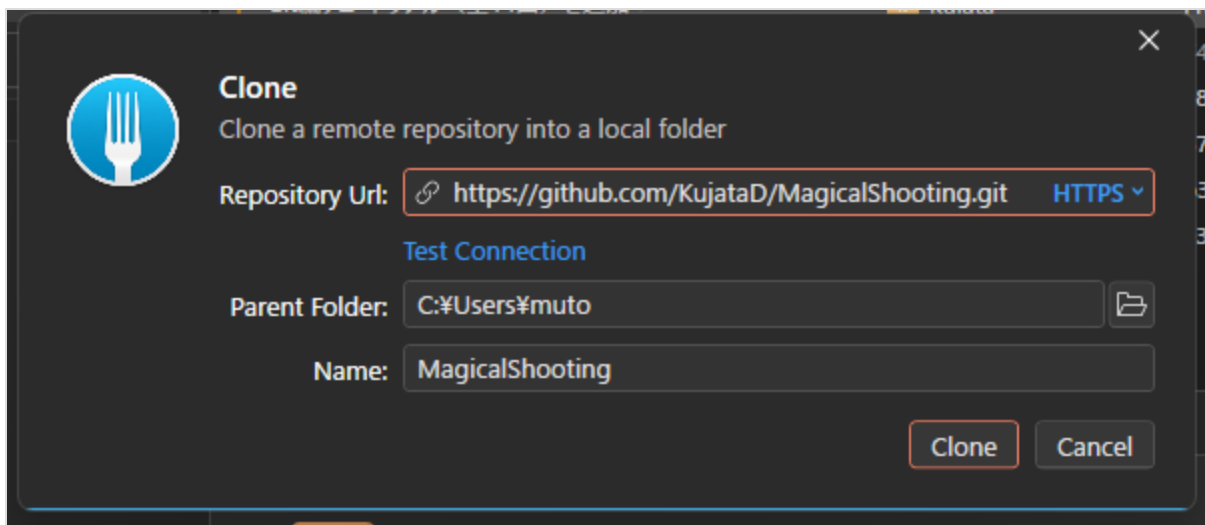
GitHub上のリポジトリを、まるごと自分のPCにコピーすること。ファイルをダウンロードするのとの違いは、変更履歴も一緒にコピーされて、GitHubとつながった状態になることです。

3. ※ここからはチーム全員がやる作業です。代表の人も含めて全員、自分のPCに持ってきてみましょう。
4. まず、GitHubのリポジトリページを開いて、緑色の「Code」ボタンを押します。
出てきたURL（`https://github.com/...` で終わるもの）の右にあるコピーアイコンを押して、URLをコピーします。



リポジトリのCodeボタン

5. 次にForkを開いて、File > Clone を選びます。



ForkのCloneダイアログ

6. 入力するのは2つです。

- **Repository Url** : さっきコピーしたURLを貼り付け
- **Parent Folder** : どのフォルダの中に置くか (例 : C:\Users\自分の名前\source\repos)

※Nameは自動で入ります。Parent Folderに指定したフォルダの中に、リポジトリ名のフォルダが自動で作られます。あらかじめ空フォルダを作っておく必要はありません。

7. 【重要】置き場所の注意

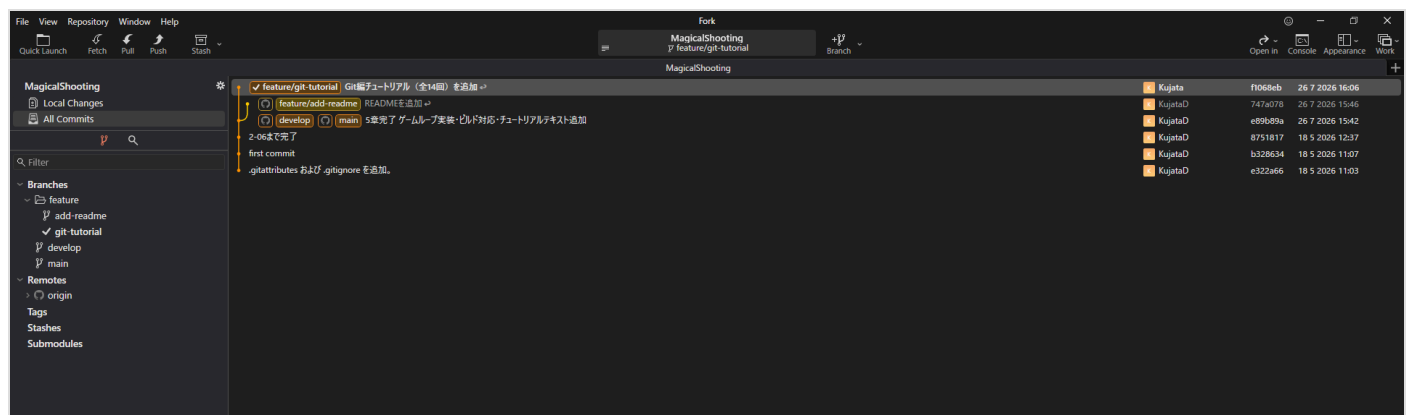
Parent Folderに指定するフォルダは、次の条件を守ってください。

- パスに日本語やスペースを含めない (C:\Users\山田\デスクトップ などNG)
- OneDriveやGoogle Driveの同期フォルダの中に置かない ← これが特に重要 !

後者は事故が多いです。GitとOneDriveが同時に同じファイルを書き換えようとして、両方おかしくなります。バックアップはGitHub側で取れているので、クラウド同期は不要です。おすすめは C:\Users\自分の名前\source\repos あたりです。

8. 「Clone」ボタンを押すと、ダウンロードが始まります。プロジェクトの大きさによっては数分かかります。

9. 完了すると、Forkの画面に変更履歴が表示されます。ここが今後の作業画面になります。



Forkのメイン画面

画面の見方は3ブロックだけ覚えておけば大丈夫です。

- **左**：ブランチ（作業場所）の一覧
- **中央**：変更履歴の一覧
- **下**：選択した履歴の中身

10. では、UnityHubからこのプロジェクトを開いてみましょう。

- UnityHubの「Add」→「Add project from disk」
- さっきクローンしたフォルダを選択

初回はLibraryフォルダを一から作り直すので、**5～15分ほどかかります**。これが1-05で説明した「Unityが自動生成するデータを各自のPCで作る」作業です。待ちましょう。

11. Unityでプロジェクトが開けたら、今回は完成です！

準備はここまで。次回からは、実際に作業して記録していきます。

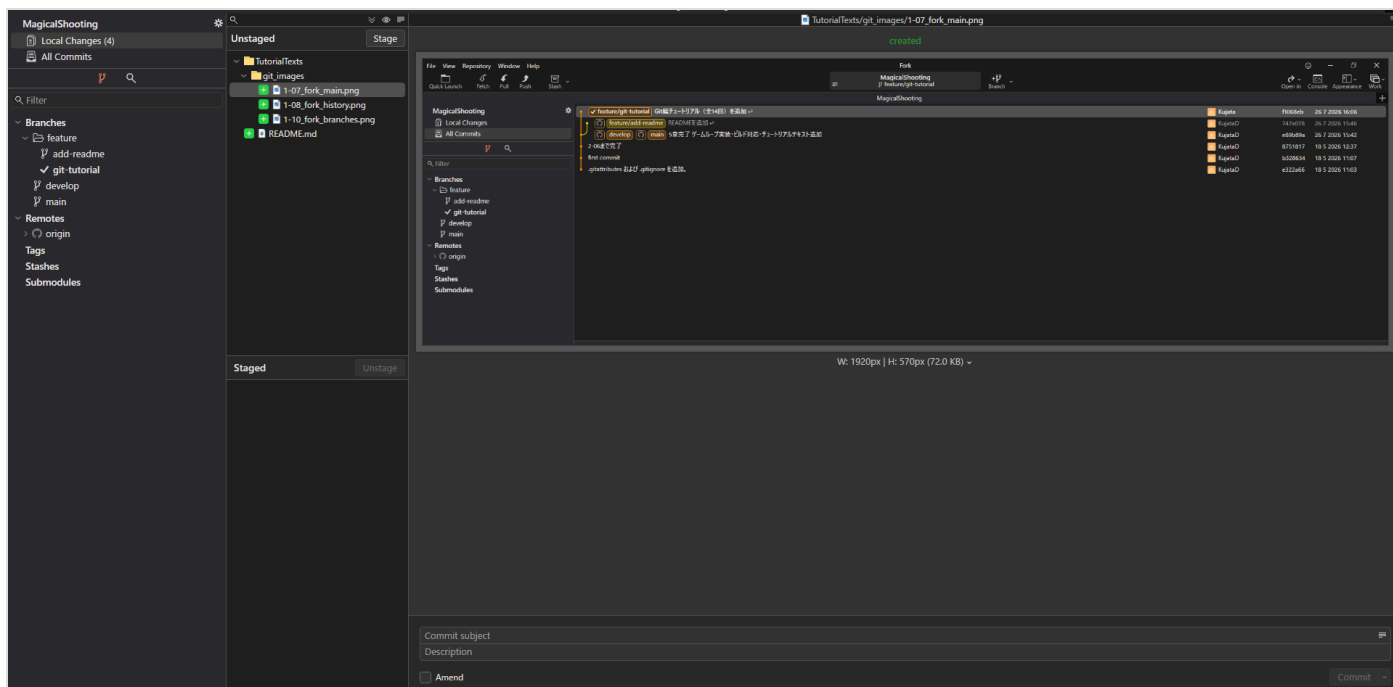
【1-08】コミットしよう（変更を記録する）【1章：Git導入編】

1. クリア条件：ファイルを変更して、コミットを1回できればOK！
2. いよいよGitの基本操作、コミットです。

用語：コミット（commit）

その時点のファイルの状態を、変更履歴として記録すること。ゲームのセーブに近いですが、上書きではなく、記録するたびにセーブデータが1つずつ増えていくタイプです。だから何度でも過去の状態に戻れます。

3. まずは何か変更を作りましょう。練習なので内容は何でもOKです。
（例：シーンにテキストを1つ追加して保存、画像を1枚Assetsに追加、など）
※必ず**Unity側でCtrl+Sして保存**してから、Forkに戻ってください。保存していない変更はGitからは見えません。
4. Forkを開くと、左メニューの一番上に「Local Changes」という項目があり、数字が付いているはずです。この数字が「変更されたファイルの数」です。クリックしてみましょう。



ForkのLocal Changes画面

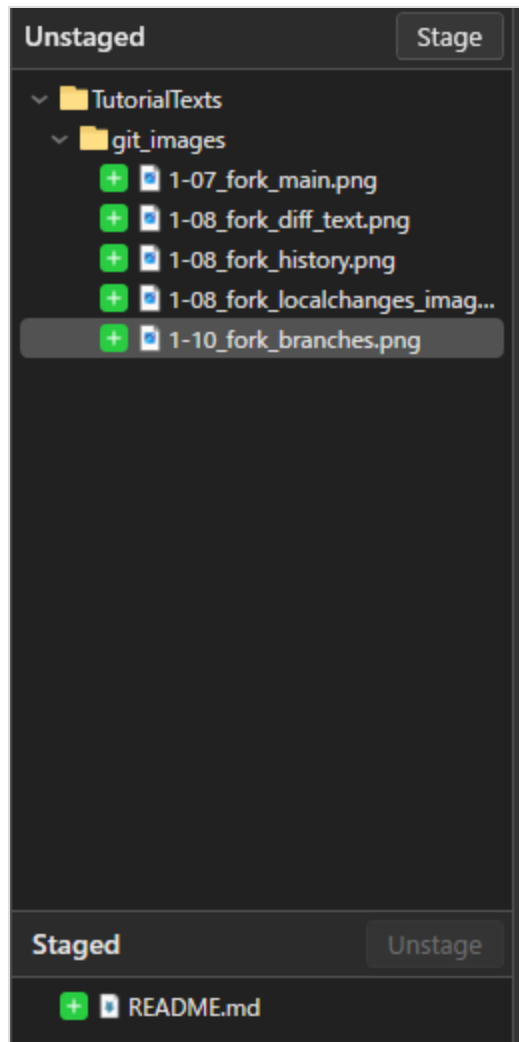
画像ファイルを選ぶと、右側にプレビューが表示されます。素材を差し替えたときの確認に便利です。

5. 画面が上下2つに分かれています。ここがコミットの操作画面です。
 - 上（Unstaged）：変更されたけど、まだ記録の対象に選んでいないファイル
 - 下（Staged）：記録の対象に選んだファイル

用語：ステージング

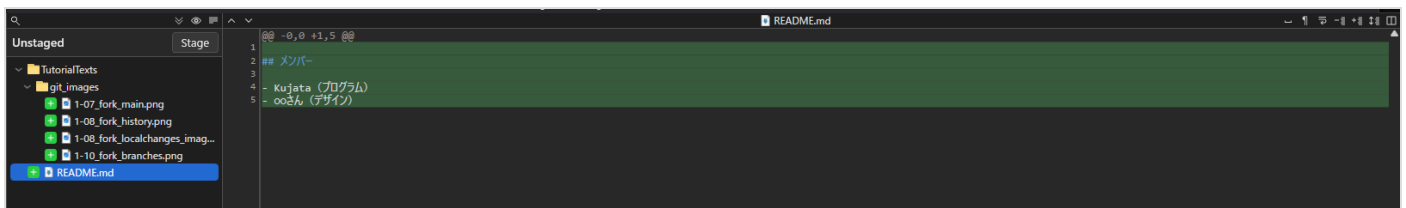
「この変更を今回の記録に含める」と選ぶ作業のこと。変更したファイル全部を毎回記録する必要はなく、**今回の作業に関係するファイルだけ**を選べます。

6. ファイルを選んで「Stage」ボタン（または右クリック→Stage）を押すと、下に移動します。
全部まとめて記録するなら、全選択して「Stage」でOKです。



StagedとUnstaged

※ファイルを選ぶと、右側にそのファイルの変更内容が表示されます。テキストファイルなら、**追加した行が緑色**で表示されます。



テキストの差分表示

7. 【1-05の復習】.metaファイルも一緒にステージしてください。
画像を1枚追加すると「Player.png」と「Player.png.meta」の2つが出てくるはずですが、**必ずセット**で記録します。片方だけだと、相手の環境で絵が外れます。
8. 下の入力欄に、コミットメッセージを書きます。ここが実は一番大事です。

用語：コミットメッセージ

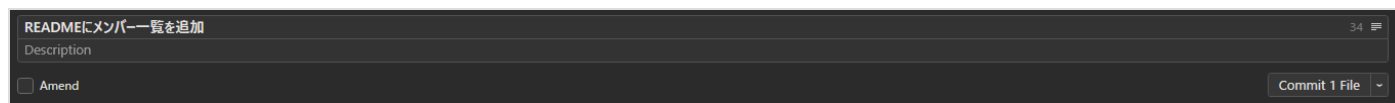
「この変更で何をしたか」を書く一言メモ。あとから履歴を見たとき、**自分とチームメンバーがこれだけを頼りに中身を判断します。**

9. 良いメッセージと悪いメッセージの例を挙げます。

✖ 悪い例	○ 良い例
修正	敵キャラの当たり判定を小さく修正
aaa	タイトル画面のロゴ画像を差し替え
いろいろ	プレイヤーの移動速度を5.0に調整
(空欄)	ゲームオーバー時のBGMを追加

コツは「何を」「どうしたか」を書くことです。3ヶ月後の自分は「修正」の中身を覚えていません。

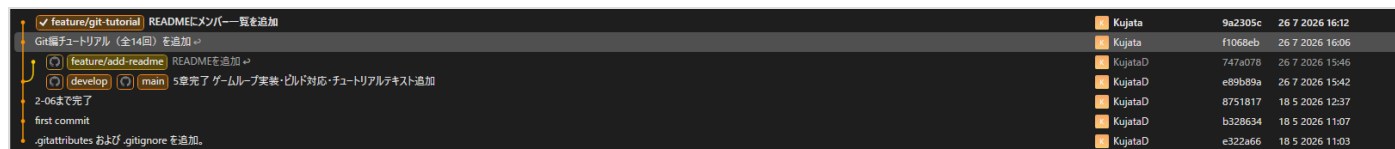
10. メッセージを書いたら「Commit」ボタンを押します。これで記録完了です！



コミットメッセージの入力欄

※ボタンに「Commit 1 File」のように対象ファイル数が表示されます。意図した数になっているか確認してから押しましょう。

11. 左メニューの「All Commits」（変更履歴）を見てみましょう。今のコミットが、名前とメッセージ付きで一番上に追加されているはずです。



Forkのコミット履歴

左端のカラフルな線は、ブランチ（作業場所）の枝分かれを表しています。今は意味が分からなくてOKです。1-10で扱います。

12. 【コツ】コミットは細かく、こまめに

「1日の終わりにまとめて1回」ではなく、「1つの作業が終わるたびに1回」がおすすめです。

- ✖ 「今日やったこと全部」を1回で記録（戻れる場所が粗くなる）
- 「敵の絵を追加」「敵のサイズ調整」「敵の色味修正」と3回に分ける（ピンポイントで戻れる）

13. コミットが履歴に表示されたら、今回は完成です！

.....ただし、これはまだ**自分のPCの中だけ**の記録です。チームにはまだ何も届いていません。次回、GitHubに送ります。

【1-09】プッシュ & プルしよう【1章：Git導入編】

1. クリア条件：自分のコミットをGitHubに送って、メンバーの変更を受け取れればOK！

2. 前回のコミットは、**自分のPCの中だけ**の記録でした。

Gitでは「自分のPCの記録」と「GitHub上の記録」が別々に存在していて、この2つは自動では同期しません。自分のタイミングで送ったり受け取ったりします。

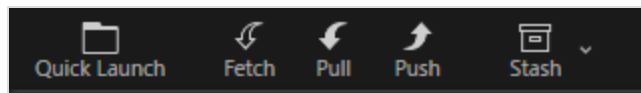
3. 今回覚える操作は2つ。送るか、受け取るだけです。

操作	向き	何をする
プッシュ (push)	自分のPC → GitHub	自分のコミットをGitHubに送る
プル (pull)	GitHub → 自分のPC	メンバーのコミットを自分のPCに取り込む

用語：プッシュ / プル

push は「押し出す」、pull は「引っばる」。自分から見て、送り出すのか、引き寄せるのかで覚えると迷いません。

4. まずプッシュしてみましょう。Forkの上部にある「Push」ボタン（↑の矢印アイコン）を押します。

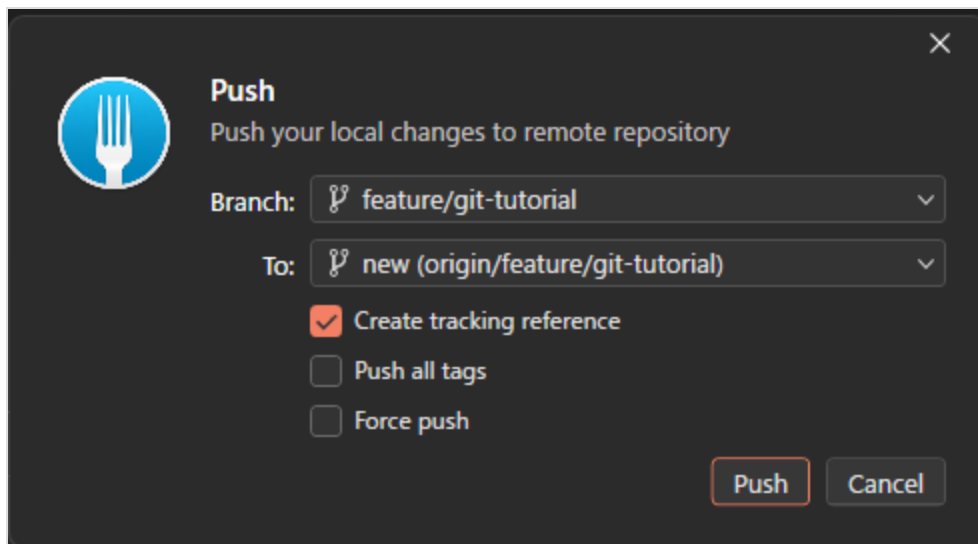


Forkのツールバー

※左から Quick Launch / Fetch / **Pull** / **Push** / Stash。今回使うのは3番目と4番目です。

5. 確認ダイアログが出るので、そのまま「Push」を押せばOK。

数秒待ってエラーが出なければ送信完了です。



Pushダイアログ

※「Create tracking reference」にチェックが入っていればそのままOK。「このブランチをGitHub側と紐づける」という意味です。
※初回はGitHubのログインを求められることがあります。1-04で連携していれば自動で通ります。

6. GitHubのリポジトリページをブラウザで開いて、リロードしてみてください。

さっきのコミットメッセージが表示されているはずです。これでチーム全員があなたの作業を見られる状態になりました。

7. 次はプル。メンバーが送った変更を受け取る操作です。

同じツールバーの「Pull」ボタン（↓の矢印アイコン）を押すだけ。こちらも確認ダイアログはそのままOKで大丈夫です。

8. 【超重要】作業を始める前は、必ずプルする

これはチーム制作で一番大事な習慣です。理由はこうです。

× プルしないで作業した場合

Aさん：古い状態のまま作業 → プッシュしようとする

GitHub：「その間に他の人が更新しているよ」→ 拒否される

Aさん：手作業でまとめ直すことになる

○ 作業前にプルした場合

Aさん：最新を取得 → 作業 → プッシュ（そのまま通る）

Unityを開く前に、まずForkでPull。これを毎回の習慣にしてください。

9. 【Unity特有の注意】プルする前に、Unityは閉じておく

Unityを開いたままプルすると、Unityが開いているファイルが裏で書き換わってしまい、表示がおかしくなったりプロジェクトが壊れたりすることがあります。

安全な手順はこれです。

1. Unityで作業 → 保存 → **Unityを閉じる**

2. Forkでプル

3. Unityを開き直す

※慣れれば開いたままで大丈夫な場面も分かってきますが、最初は必ず閉じましょう。

10. 【エラーが出たら】「Push rejected」と表示された場合

「あなたが作業している間に、他の人が先に送ったよ」という意味です。慌てなくて大丈夫。

1. まず **Pull** する（相手の変更を受け取る）

2. 自動でまとめられれば、そのまま **Push** し直す

3. 自動でまとめられなかった場合（コンフリクト）は、1-13で対処法を扱います

11. プッシュしてGitHub上で確認できて、プルもできたら、今回は完成です！

これでGitの基本操作（コミット・プッシュ・プル）がそろいました。次回からは、チームで作業がぶつからないための仕組みに入ります。

【1-10】ブランチってなに？【1章：Git導入編】

1. クリア条件：ブランチを1本作って、切り替えられればOK！
2. ここまでで、1人での作業はできるようになりました。でもチーム全員が同じ場所に送り続けると、こうなります。

- Aさんが作業途中の「まだ動かない敵キャラ」を送る
- Bさんがそれを受け取る → **ゲームが起動しなくなる**
- Bさん「動かないんだけど！」
- Aさん「ごめん、作業途中で.....」

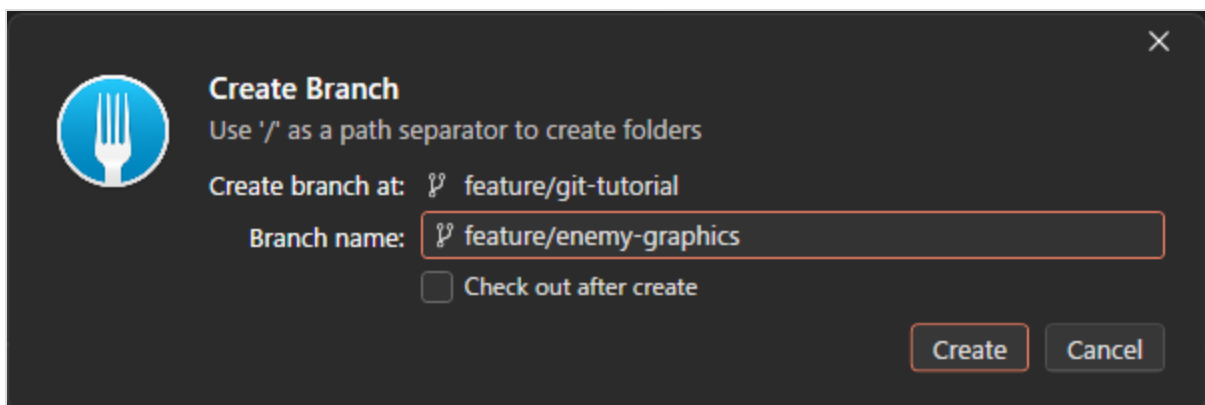
これを防ぐのがブランチです。

3. ブランチとは、他の人に影響を与えずに作業できる、独立した作業場所のことです。

用語：ブランチ（branch）

直訳は「枝」。元の状態から枝分かれさせて作る作業用のコピーのこと。枝の中で何をしても、元の状態や他の人の枝には影響しません。作業が終わってから、元の場所に合流させます。

4. 具体的には、こういう使い分けをします。
 - **main**：完成している状態を置いておくブランチ。ここは常に動く状態を保つ
 - **feature/〇〇**：自分の作業用のブランチ。ここで好きなだけ試して、途中で壊れていてもOK作業が完成したら、featureブランチの内容をまとめて合流させます。詳しいルールは次回（1-11）決めます。
5. では実際に作ってみましょう。Forkの左メニューに「Branches」という項目があり、今は **main** だけがあるはずです。
6. ブランチを作るには、画面右上の「Branch」ボタン（またはメニューの **Repository > Create Branch**）を押します。



ブランチ作成ダイアログ

- ※「Create branch at」は「どこから枝分かれさせるか」の指定です。今いるブランチが自動で入ります。
- ※「Check out after create」にチェックを入れると、作成と同時にそのブランチに移動します。入れておくと便利です。

7. 名前を付けます。ここでルールがあります。

例：

- feature/enemy-graphics （敵の絵を作る）
- feature/title-ui （タイトル画面のUI）
- feature/bgm （BGM追加）

「/」で区切ると、Forkの画面上でフォルダのようにまとまって表示されて見やすくなります。日本語も使えますが、トラブルの原因になるので**半角英数**にしてください。

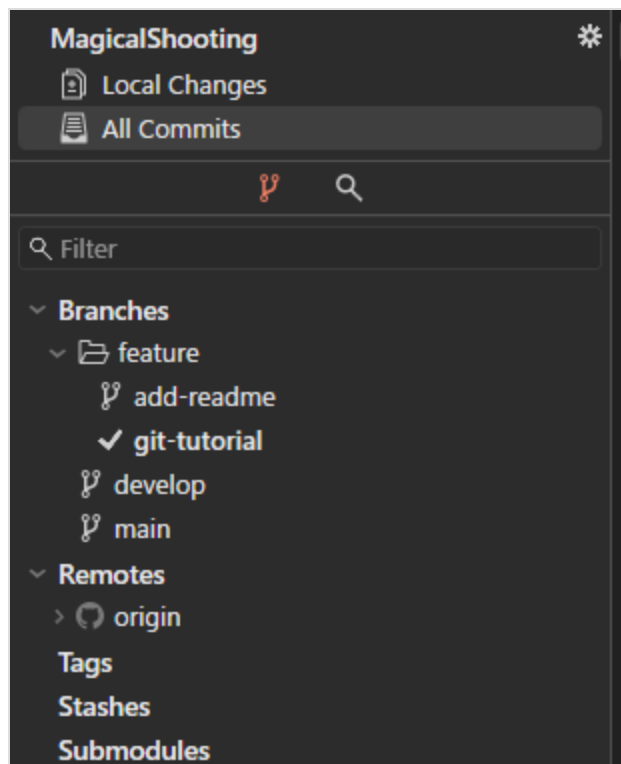
8. 「Create Branch」を押すと、新しいブランチができて、**自動でそちらに切り替わります**。

用語：チェックアウト（checkout）

作業するブランチを切り替えること。切り替えると、プロジェクトフォルダの中身が、そのブランチの状態に入れ替わります。Gitが履歴から復元しているだけなので、ファイルが消えたわけではありません。

9. 【重要】今どのブランチにいるか、必ず確認するクセをつける

Forkの左メニューで、**チェックマーク（✓）**が付いているのが**現在のブランチ**です。作業を始める前にここを確認してください。「mainで作業してしまった」は、最初のうち一番多いミスです（直し方は1-14の逆引き表に載せます）。



Forkのブランチ一覧

※ feature の下に枝がぶら下がって見えるのは、feature/oo という名前の「/」でForkが自動的にフォルダ分けして表示しているためです。

10. 試しに、ブランチを行き来してみましょう。

- featureブランチで何かファイルを追加してコミット
- mainにチェックアウト（ブランチ名をダブルクリックで切り替え）
- エクスプローラーでフォルダを見ると、追加したファイルが無くなっている
- featureに戻すと、また現れる

消えたのではなく、ブランチごとに中身が切り替わっているだけです。これがブランチの感覚です。

※ブランチを切り替えるときも、Unityは閉じておきましょう（1-09と同じ理由です）。

11. ブランチを作って、切り替えができたら、今回は完成です！

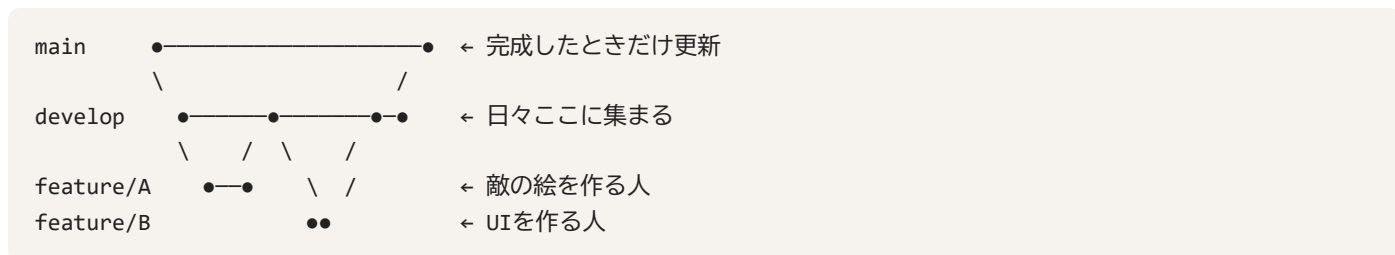
次回は、チームでブランチをどう使い分けるかのルールを決めます。

【1-11】main / develop / feature で運用しよう【1章：Git導入編】

1. クリア条件：3種類のブランチの役割が分かって、チームのルールを決められればOK！
2. 前回でブランチを作れるようになりました。ただ、全員が好きなようにブランチを作ると管理しきれなくなります。そこで、ブランチを3種類に分けて使います。これは多くの開発チームで使われている定番のやり方です。
3. 3つのブランチの役割はこうです。

ブランチ	役割	状態
main	完成版。提出・公開できる状態を保つ	常に動く
develop	開発版。全員の作業がここに集まる	だいたい動く
feature/○○	個人の作業用。1タスクにつき1本	途中で壊れていてOK

4. 図にするとこうです。作業は下から上に合流していきます。



5. ルールは3つだけです。
 - mainには直接コミットしない
 - developにも直接コミットしない（プルリクエスト経由で入れます。1-12でやります）
 - 作業は必ずfeatureブランチを作ってから

「じゃあどこで作業するの？」→ 常にfeatureです。これだけ覚えてください。

6. 【なぜmainを直接触らないのか】

提出日に「今の状態をとりあえず出そう」となったとき、mainが常に動く状態になっていれば、そこから出せば済みます。逆にmainが壊れていると、提出直前に慌てることになります。mainは最後の避難先として、きれいなまま置いておきます。

7. featureブランチの名前は、作業内容が分かるように付けます。

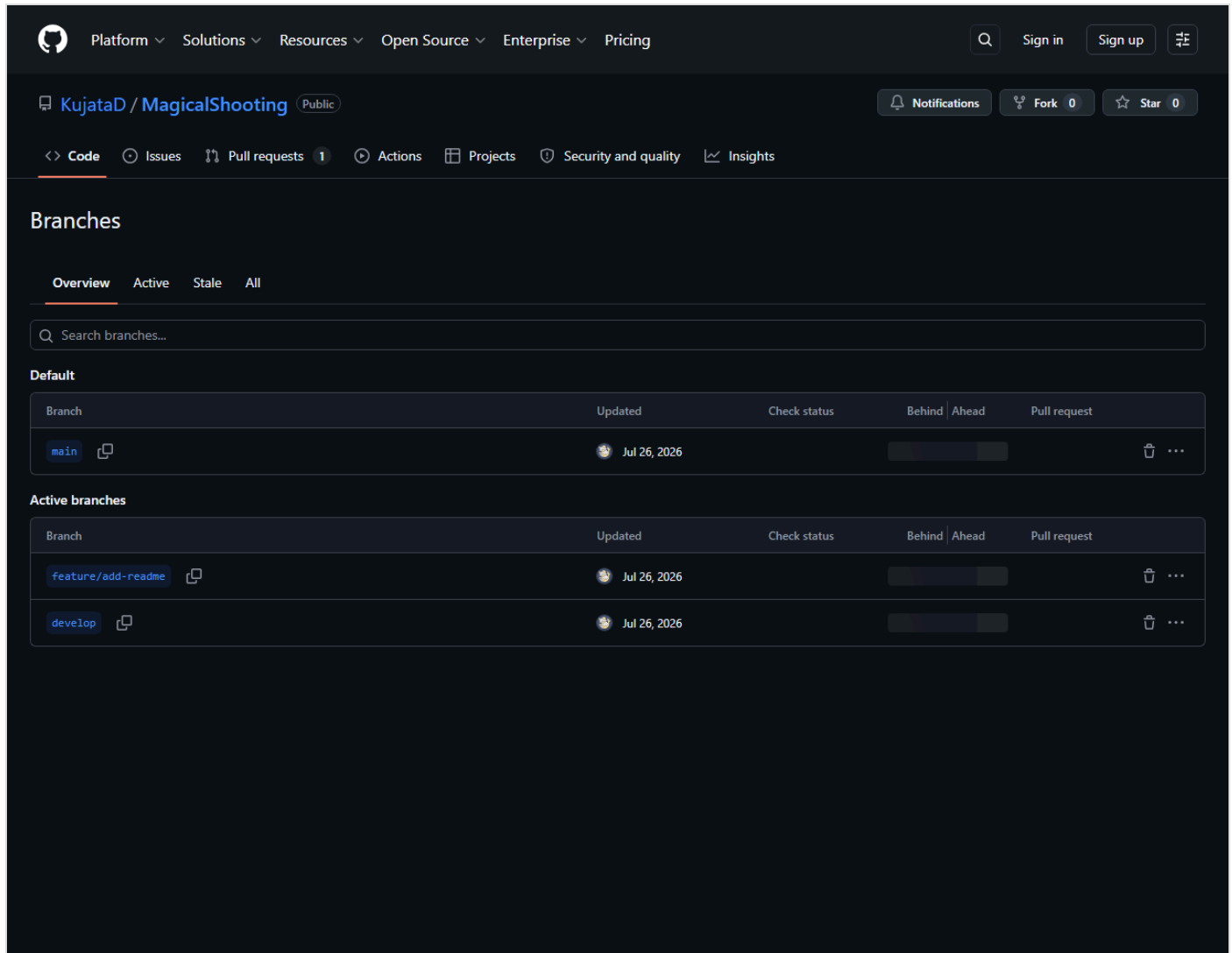
feature/enemy-graphics	敵キャラの絵
feature/title-ui	タイトル画面
feature/bgm-stage1	ステージ1のBGM
feature/fix-player-speed	プレイヤー速度の調整

ポイントは「1つのブランチ＝1つの作業」。

「今日やること全部」を1本に詰め込むと、あとで中身を確認する人が大変になります。半日～2日で終わるくらいの大きさが

目安です。

8. では実際に、チームのdevelopブランチを作りましょう（代表の人が1回だけやればOK）。
 - Forkで `main` にいることを確認
 - Branch（新規作成）→ 名前を `develop` にして作成
 - **Push** して、GitHub側にもdevelopを作る
9. GitHubのリポジトリページで「Branches」を見ると、mainとdevelopの2本があるはずです。



ブランチ一覧

10. 【代表の人向け・おすすめ】mainを保護する

GitHubには「このブランチには直接送れないようにする」設定があります。ミスを仕組みで防げるので、設定しておくとう安心です。

- リポジトリの `Settings > Branches`
- 「Add branch ruleset」または「Add rule」
- Branch name pattern に `main` を入力
- 「Require a pull request before merging」にチェック

これで、うっかりmainに直接送ろうとしても、GitHubが止めてくれます。

11. 【メンバー全員でやること】自分の作業ブランチを作る

developができたら、各自こうします。

1. Forkで `develop` にチェックアウト
2. **Pull**（最新のdevelopにする）
3. Branch（新規作成）で `feature/自分の作業名` を作成
4. そこで作業開始！

※featureは**必ずdevelop**から枝分かれさせてください。mainから作ると、他の人の作業が入っていない古い状態から始まってしまいます。

12. developブランチができて、各自がfeatureブランチを持てたら、今回は完成です！

今回は、featureで終わった作業をdevelopに合流させる手続きをやります。

【1-12】プルリクエストを出そう【1章：Git導入編】

1. クリア条件：featureブランチからdevelopへのプルリクエストを1本作ればOK！
2. featureブランチでの作業が終わりました。これをdevelopに合流させたいのですが、黙って合流させると他のメンバーは気づけません。
そこで、「この作業をdevelopに入れていいか」をチームに確認してもらう手続きを踏みます。これがプルリクエストです。

用語：プルリクエスト（Pull Request、略してPR）

「自分の作業を取り込んでください」という依頼のこと。GitHub上に専用のページが作られ、そこで変更内容の確認とコメントのやり取りができます。OKが出たら合流させます。

3. 流れはこうなります。
 1. 自分のfeatureブランチで作業を完成させる
 2. すぐdevelopに入れず、まずPRを作る
 3. 他のメンバーが内容を見て、気になる点があればコメントする
 4. 問題なければ合流させる（次回1-13でやります）

※このサークルでは全員が同じようにUnityとスクリプトを触ります。レビューする側も同じメンバーなので、「専門家の審査」ではなく「お互いの確認」だと思ってください。

4. まず、作業をプッシュします。1-09でやった操作です。
Forkで自分のfeatureブランチにいることを確認して、コミット → **Push**。

※初回のプッシュ時は「Publish branch」のような表示になります。GitHub側にまだこのブランチが無いので、新しく作るという意味です。そのままPushでOK。

5. プッシュしたら、GitHubのリポジトリページをブラウザで開いてください。
黄色い帯で「feature/○○ had recent pushes」と表示され、「Compare & pull request」という緑のボタンが出ているはずです。これを押します。

※出ていない場合は、上部メニューの「Pull requests」→「New pull request」からでもOKです。

6. 【最重要】合流先（base）の確認

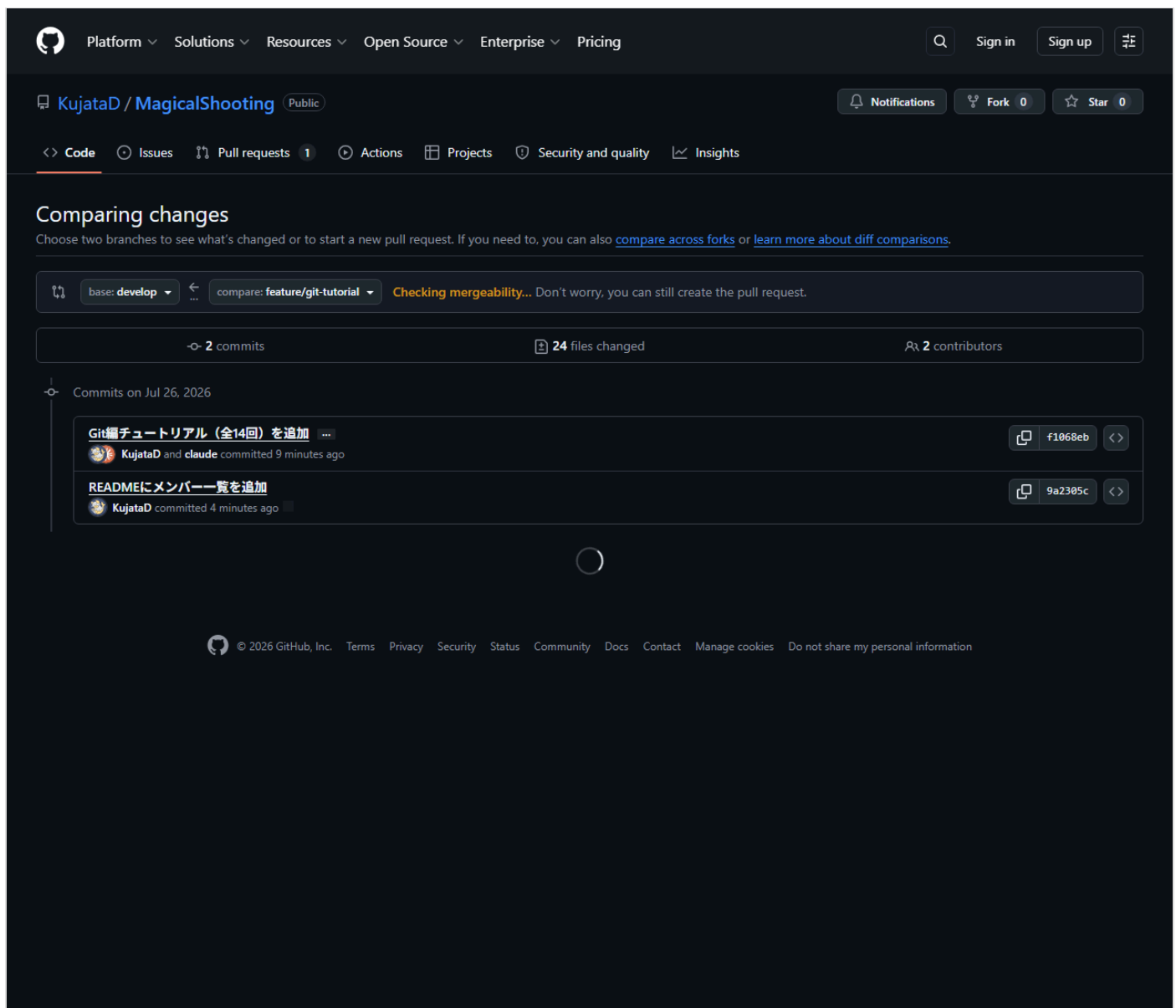
PR作成画面の上部に、こういう表示があります。

```
base: main ← compare: feature/enemy-graphics
```

ここのbaseを **develop** に変更してください。

初期状態ではmainになっていることが多いですが、1-11のルール通り、featureはdevelopに合流させます。

```
base: develop ← compare: feature/enemy-graphics ○ これが正解
```



PR作成画面（baseがdevelopになっている）

7. タイトルと説明を書きます。

- **タイトル**：何をしたか一言（例：「敵キャラのグラフィックを追加」）
- **説明**：もう少し詳しく

説明はこのテンプレートに沿って書けば十分です。

やったこと

- 敵キャラ（お化け）の画像を3種類追加
- Enemyプレハブの見た目を差し替え

確認してほしいこと

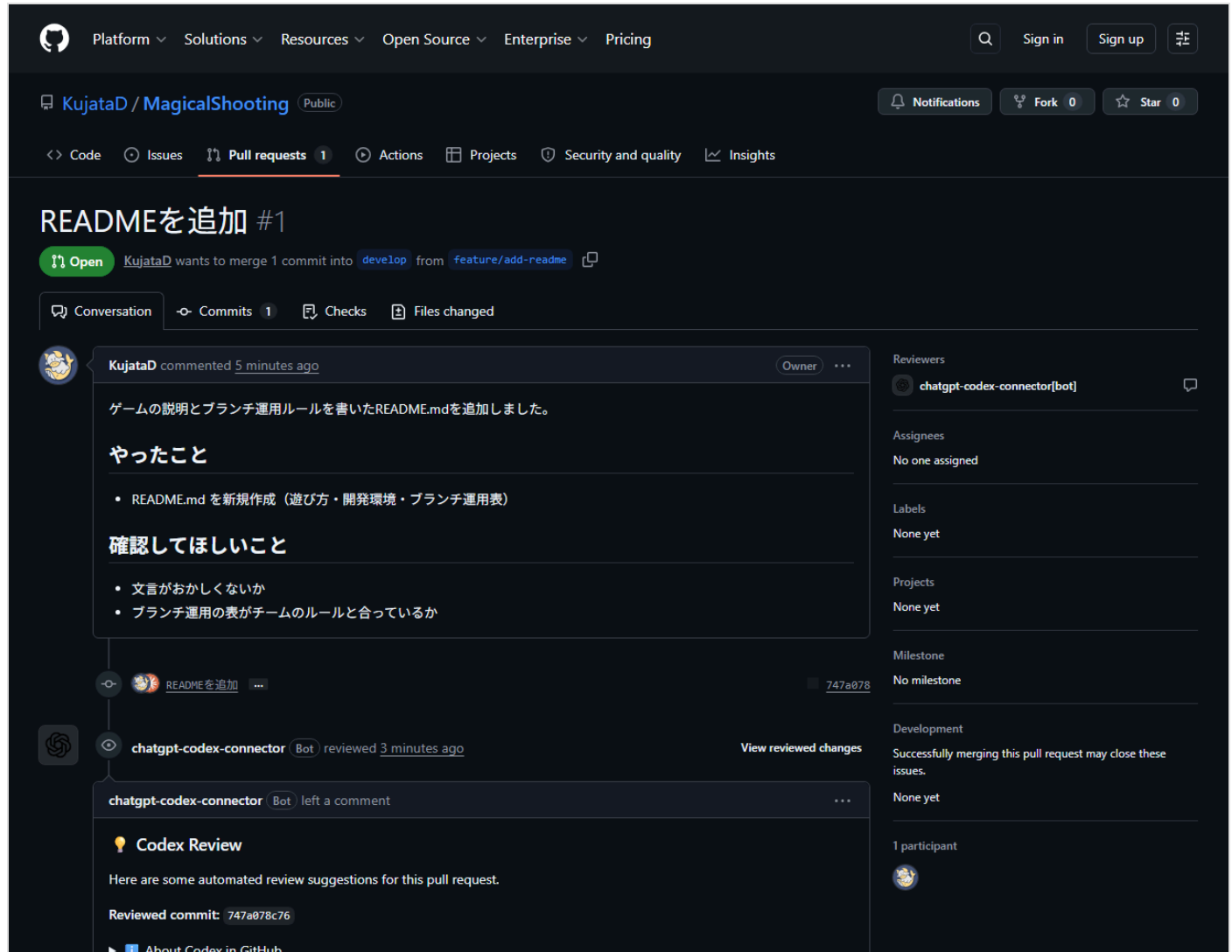
- サイズ感が他のキャラと合っているか
- 色味が背景と喧嘩していないか

スクショ

（ここに画像をドラッグ&ドロップで貼れます）

※GitHubの説明欄は、画像をドラッグ&ドロップでそのまま貼れます。見た目に関わる変更のときは、変更前後のスクショを貼っておくと、確認する人がゲームを起動しなくても判断できます。

8. 「Create pull request」を押せば、PRの完成です！

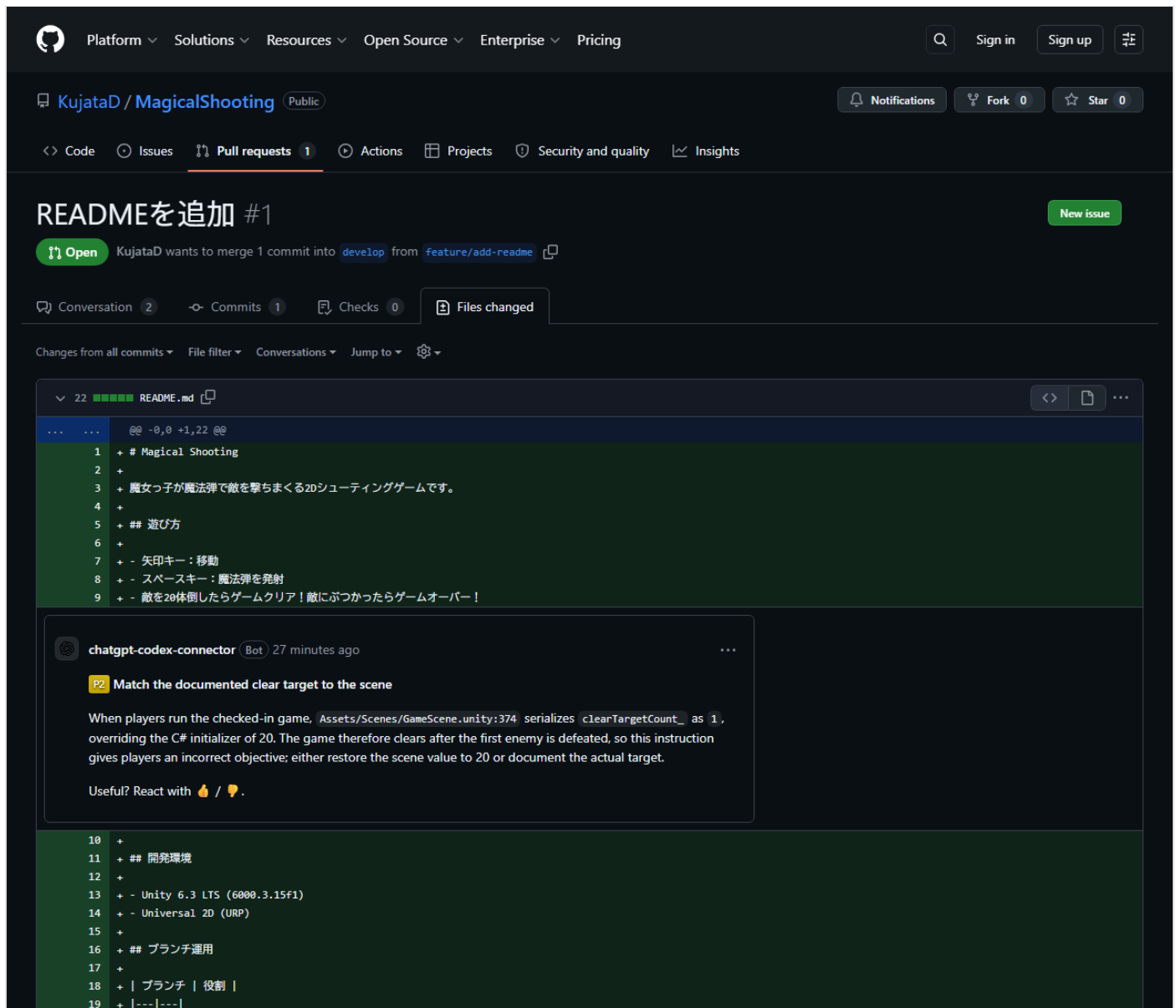


プルリクエストの画面

9. PR画面には3つのタブがあります。

- **Conversation**：コメントのやり取り。ここで相談します
- **Commits**：このPRに含まれるコミットの一覧
- **Files changed**：変更されたファイルの中身が全部見られるタブ。画像は変更前後を並べて表示できます

確認するときのメイン画面は「Files changed」です。行にマウスを乗せると「+」が出て、その行に対してピンポイントでコメントを付けられます。



Files changedタブとコメント

※このスクショでは、自動チェックのBotが「READMEには敵20体でクリアと書いてあるが、シーンの設定は1体になっている」と指摘しています。こういう食い違いを合流前に見つけられるのが、PRを挟む一番のメリットです。

10. PRを出したら、チームに一声かけましょう。

> 「敵の絵のPR出しました！確認お願いします 🙏」

GitHubからも通知は飛びますが、埋もれやすいのでDiscordなどで伝えるのが確実です。

11. 【確認する側になったら】

難しく考えなくて大丈夫です。見るのはこの3点だけ。

- 意図した変更だけが入っているか（関係ないファイルが混ざっていないか）
- 明らかにおかしいところがないか
- 自分が今触っているファイルと重なっていないか

問題なければ「Files changed」→「Review changes」→「Approve」でOKサインを出します。

気になる点があれば、コメントを残して「Request changes」。**分からない部分は「ここどうなってる？」と質問するだけでも十分**です。全員が同じくらいの経験値なので、遠慮なく聞きましょう。

12. PRが作れて、確認をお願いできたら、今回は完成です！

次回、いよいよ合流させます。

【1-13】マージとコンフリクト【1章：Git導入編】

1. クリア条件：PRをマージできて、コンフリクトが起きたときの直し方が分かればOK！
2. 確認でOKが出ました。いよいよ合流させます。この合流作業がマージです。
3. GitHubのPR画面を下にスクロールすると、緑色の「Merge pull request」ボタンがあります。これを押して、確認の「Confirm merge」を押すだけで完了です。
※ボタンが緑なら「自動で合流できます」という意味。灰色や赤の場合は、後半で説明するコンフリクトが起きています。
4. マージが終わると、「Delete branch」というボタンが出ます。押して大丈夫です。

用語：ブランチの削除

役目を終えたfeatureブランチを消すこと。作業内容が消えるわけではありません。中身はdevelopに合流済みで、履歴にも残っています。消すのは使い終わった枝だけです。放置するとブランチ一覧が増えすぎるので、こまめに消しましょう。

5. マージ後、チーム全員がやる必要があります。
> developをプルして、最新にする
誰かのPRがマージされると、他のメンバーの手元のdevelopは古い状態になります。Discordで「マージしました」と伝えて、全員がプルする。ここまでで1サイクルです。
6. ここからが本題。コンフリクトの話です。

用語：コンフリクト (conflict)

直訳は「衝突」。2人が同じファイルの同じ場所を別々に書き換えたときに起きる、Gitが自動でまとめられない状態のこと。

7. なぜ起きるのか。Gitはこう判断しています。

○ 自動でまとめられる

Aさん：3行目を修正

Bさん：50行目を修正

→ 場所が違うので、両方採用すればいい

× まとめられない (コンフリクト)

Aさん：3行目を「赤」に変更

Bさん：3行目を「青」に変更

→ どちらが正しいか、Gitには判断できない

Gitはどちらが正しいかまでは分かりません。だから「人間が決めてください」と処理を止めます。故障ではなく、正常な動作です。

8. 【予防が9割】そもそも起こさない工夫

コンフリクトは、直すより起こさない方がずっと楽です。1-05でも触れましたが、大事なので再掲します。

- 同じシーン・同じプレハブを2人で同時に触らない（Unityで最も起きやすい）
- 作業前にDiscordで「今日はTitleScene触ります」と宣言する
- featureブランチは小さく、短期間で終わらせる（長く持っているほど衝突しやすい）
- 作業開始前に必ずdevelopをプルする

9. 【起きてしまったら】GitHub上で解決する（簡単なケース）

PR画面に「This branch has conflicts that must be resolved」と出たら、「Resolve conflicts」ボタンを押します。編集画面が開いて、こういう記号が表示されます。

```
<<<<<< feature/enemy-graphics
speed = 8.0f;          ← 自分の変更
=====
speed = 5.0f;          ← 相手の変更
>>>>>> develop
```

やることは3つです。

1. 正しい方を残して、いらない方を消す（両方必要な場合は両方残す）
2. <<<<<<、=====、>>>>>> の記号3行も全部消す
3. 「Mark as resolved」→「Commit merge」

※記号の消し忘れが一番多いミスです。残したままだと、その記号がファイルに残って動かなくなります。

10. 【Unityのシーンでコンフリクトしたら】無理に直さず作り直す

正直に言うと、Unityのシーンファイルのコンフリクトは、手作業で直すものではありません。

中身は数千行の複雑なテキストで、人間が正しく直すのはほぼ不可能です。

現実的な対処はこれです。

- どちらか一方の変更を丸ごと採用する（相手と話して決める）
- 採用しなかった側の作業は、Unity上でやり直す（悔しいですが、これが一番速くて確実です）

Forkなら、コンフリクトしたファイルを右クリック → 「Use 'ours'」（自分の方を採用）または「Use 'theirs'」（相手の方を採用）で切り替えられます。

※だからこそ、手順8の予防が重要です。「同じシーンを触らない」を徹底してください。

11. 【最後の手段】どうしようもなくなったら

GitHubに送ってあるものは無事です。手元の状態がおかしくなったら、こうすればリセットできます。

1. 今のプロジェクトフォルダを別名にリネーム（念のため残しておく）
2. GitHubからもう一度クローンし直す（1-07の手順）
3. 必要なファイルだけ、古いフォルダから手作業でコピーする

「壊したかも」と思っても、プッシュ済みのものは消えません。落ち着いて対処すれば大丈夫です。

12. PRをマージできて、コンフリクトの直し方が分かったら、今回は完成です！

次回は最終回、全部忘れても大丈夫な保存版マニュアルを作ります。

【1-14】通し：チーム作業分担マニュアル【保存版】

1. クリア条件：これ1枚を見れば、内容を忘れていても作業を進められればOK！
2. 最終回です。ここまでの内容を忘れても、このページさえ開けば作業が回るようにまとめました。
印刷するなりDiscordにピン留めするなりして、手元に置いてください。

【A】毎日の作業フロー（これだけ覚えればOK）

```
① Discordで「今日はooを触ります」と宣言
↓
② Fork：develop に切り替えて Pull（最新を取得）
↓
③ Fork：Branch で feature/oo を作る
↓
④ Unity で作業する（こまめに保存）
↓
⑤ Unity を閉じる
↓
⑥ Fork：変更を Stage → コミットメッセージを書く → Commit
（④～⑥は何回繰り返してもOK。こまめに！）
↓
⑦ Fork：Push（GitHubに送る）
↓
⑧ GitHub：Pull Request を作る（base は develop ！）
↓
⑨ Discordで「PR出しました、確認お願いします」
↓
⑩ 確認OK → GitHub：Merge → ブランチ削除
↓
⑪ Discordで「マージしました」→ 全員が develop を Pull
↓
⑫ ①に戻る
```

【B】ブランチの役割（3つだけ）

ブランチ	役割	触り方
main	完成版。いつでも遊べる状態	直接触らない。提出・公開のときだけ
develop	開発版。全員の作業の集合場所	直接コミットしない。PR経由で入れる
feature/○○	個人の作業場所	ここで作業する。1作業1本

ブランチ名の例： `feature/enemy-graphics` `feature/title-ui` `feature/bgm-stage1`

【C】Unityチーム制作の決めごと（5ヶ条）

1. 同じシーン・同じプレハブを2人で同時に触らない
→ いちばん事故る。作業前にDiscordで宣言する
2. .metaファイルも必ず一緒にコミットする
→ 忘れると相手の環境で絵やスクリプトの紐付けが外れる
3. Pull / ブランチ切り替えのときは Unity を閉じる
→ 開いたままやるとUnityの表示やデータがおかしくなる
4. Libraryフォルダは共有しない
→ .gitignoreで除外済み。各自のPCで自動生成される
5. 作業前は必ず develop を Pull する
→ 古い状態から作業を始めると、あとでまとめられなくなる

【D】チームでの分担の決め方

作業がぶつからないように、ファイル単位で担当を決めるのがコツです。

良い分担 ○	避けたい分担 ×
Aさん：Enemyプレハブ周り Bさん：Playerプレハブ周り	Aさん：敵の動き Bさん：敵の見た目 → 同じファイルに触ることになる
Aさん：TitleScene Bさん：GameScene	2人で同じGameSceneを編集
素材の差し替えはAさんが一括で行う	全員が好きなタイミングで素材を追加

大きなシーンは、機能ごとにプレハブに分けておくと分担しやすくなります。

（例：UI一式をプレハブ化 → UI担当はプレハブだけ触ればよく、シーン本体は触らない）

【E】困ったとき逆引き表

こうなった	こうする
mainで作業してしまった（まだコミット前）	Forkでブランチを作って切り替える。変更はそのまま新しいブランチに引き継がれる
mainにコミットしてしまった（まだPush前）	まずFork右上の Branch で新しいブランチを作成 → そのあとmainを1つ前に戻す（該当コミットを右クリック →「Reset ... to Here」）。不安なら代表の人に相談
Pushが拒否された（Push rejected）	先にPullする → 自動でまとめられればもう一度Push

コンフリクトした	1-13参照。シーンファイルなら、片方を丸ごと採用してもう片方はUnityでやり直すのが速い
相手の環境だけ絵が消える	.metaファイルのコミット漏れ。Assetsの.metaを全部コミットし直す
プロジェクトが壊れた気がする	フォルダをリネームして、GitHubからクローンし直す。Push済みのものは消えない
間違えてPRをマージした	GitHubのPR画面に「Revert」ボタンがある。取り消し用のPRが自動で作られる
どのブランチにいるか分からない	Forkの左メニューでチェックマークが付いているのが現在地
Unityが「シーンが壊れています」と言う	Unityを閉じて、Forkで該当ファイルを「Discard changes」→ Pullし直す

【F】コミットメッセージの書き方

- 敵キャラのグラフィックを3種類追加
- タイトルロゴのサイズを調整
- プレイヤーの移動速度を5.0に変更

- × 修正
- × いろいろ
- × aaa

「何を」「どうしたか」を書く。3ヶ月後の自分に向けて書きましょう。

【G】1日の終わりチェックリスト

- ☐ 作業中のファイルは全部コミットした？
- ☐ featureブランチにPushした？（PCが壊れてもGitHubにあれば無事）
- ☐ 完成した作業はPRを出した？
- ☐ 明日触るファイルをDiscordで宣言した？

1. お疲れさまでした！これでGit編は全部完了です。

最初は用語が多くて大変だったと思いますが、実際に毎日使うのは

Pull → ブランチを作る → 作業 → Commit → Push → PR

この流れだけです。1週間も回せば手が覚えます。

2. そして何より、Gitを覚えると思いついて試せるようになります。

ブランチを作れば、他の人に影響を与えずに好きなだけ実験できて、うまくいかなければ捨てればいい。「壊したらどうしよう」と迷う時間が減ります。

良いチーム制作を！